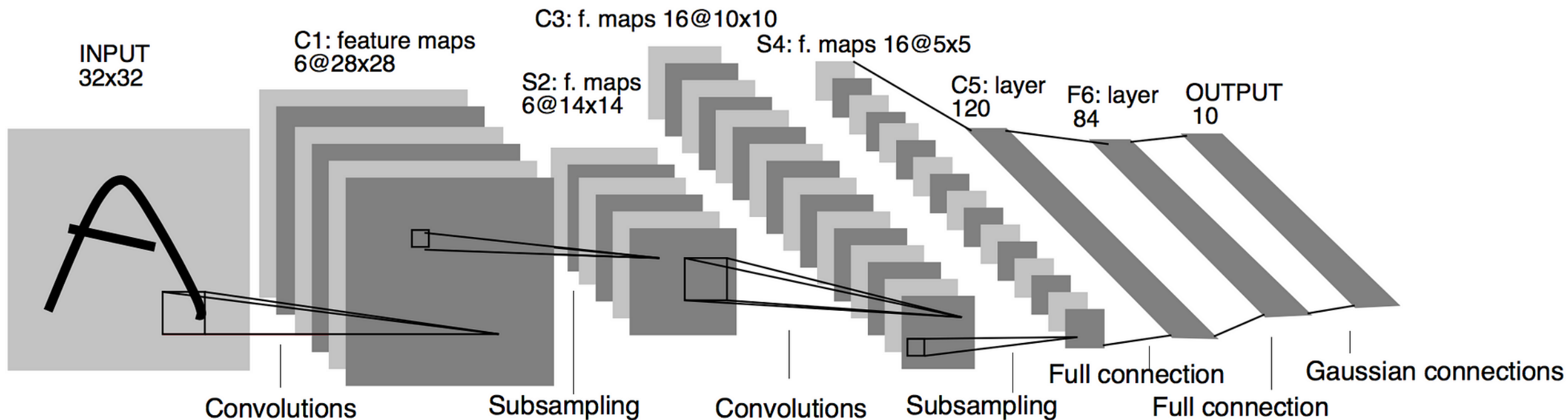
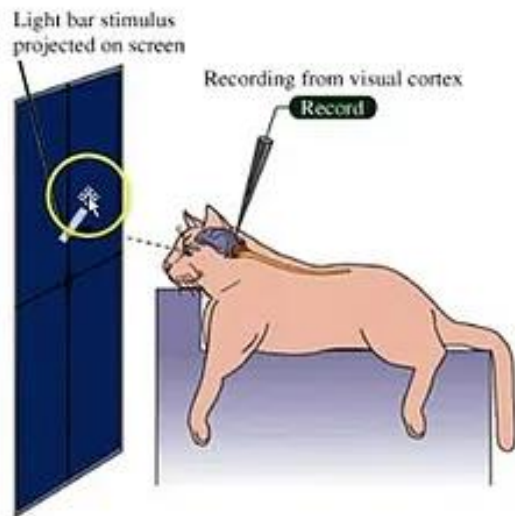


LeNet Architecture



Conv & Cat's brain

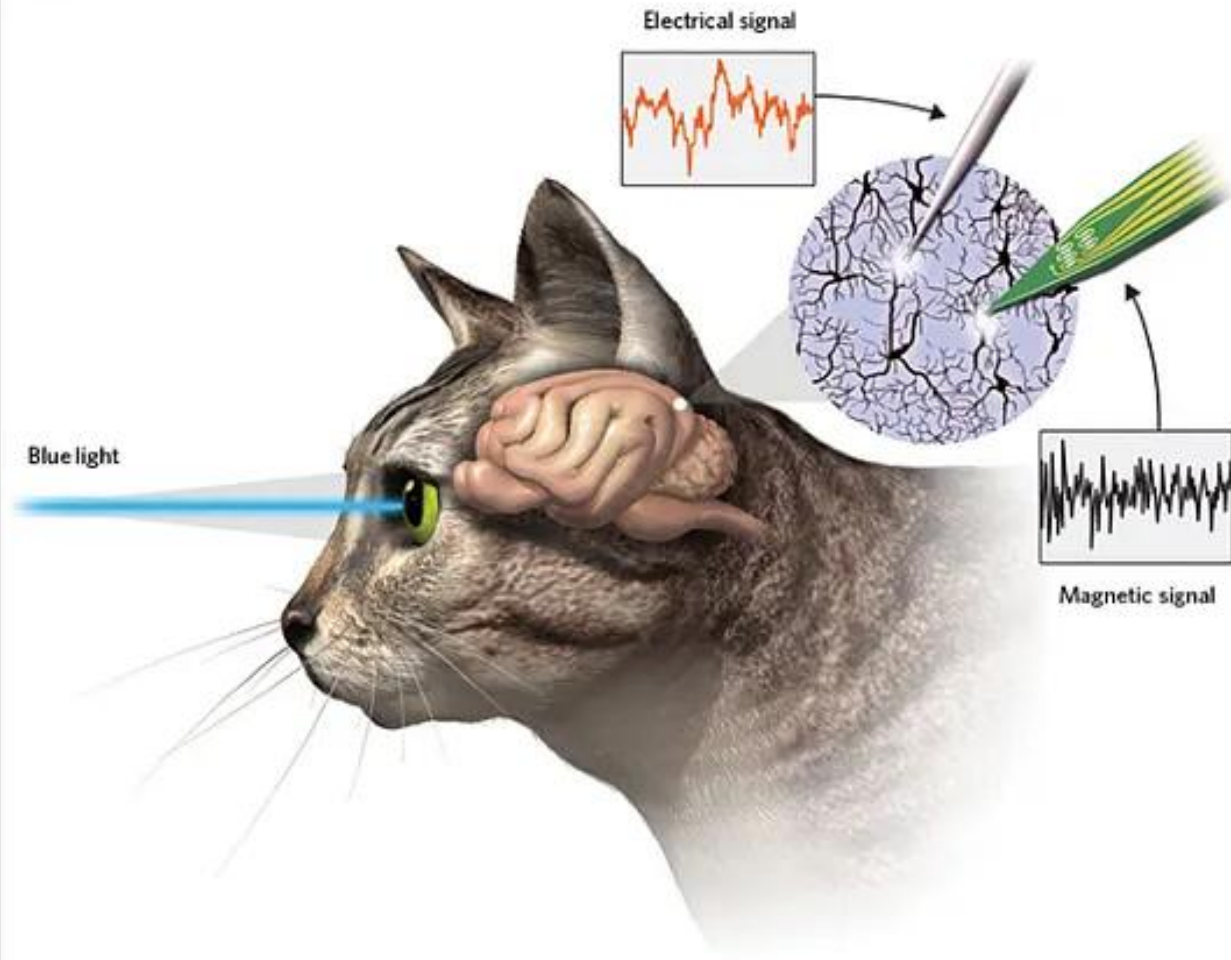
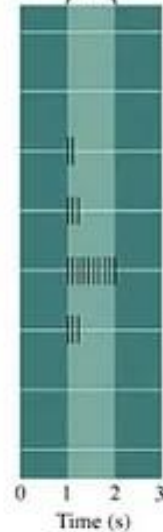
A Experimental setup



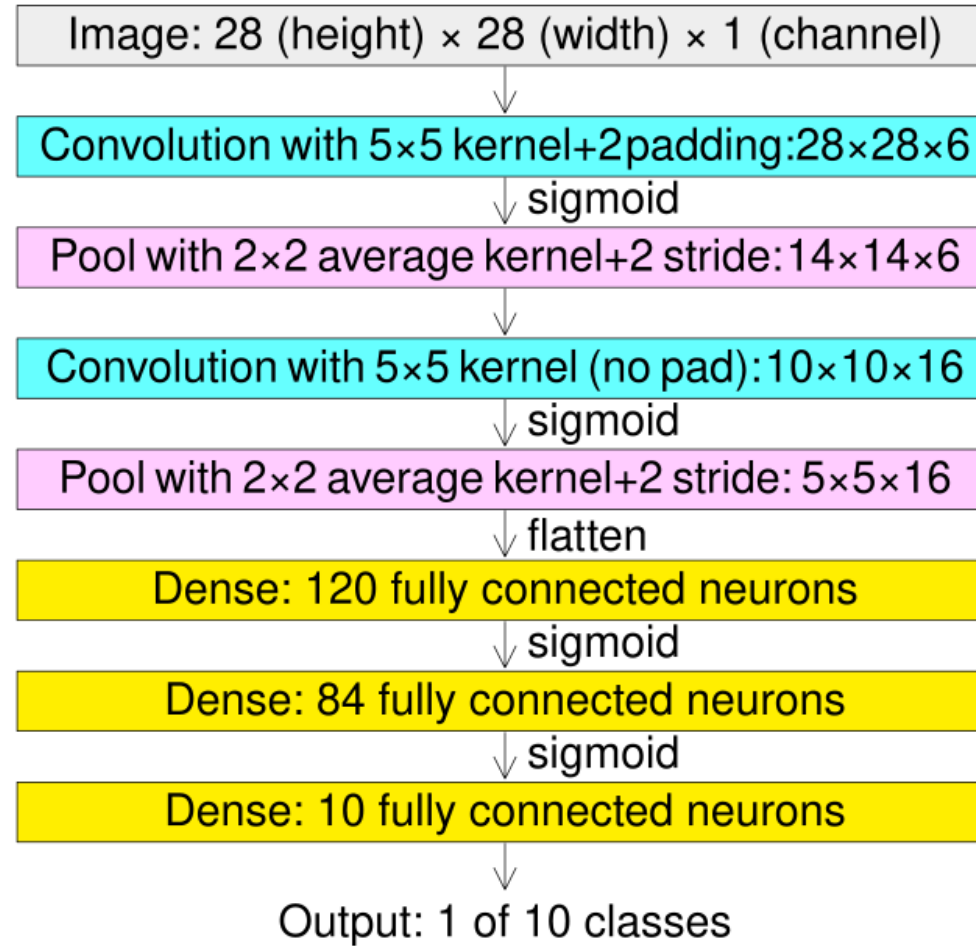
B Stimulus orientation



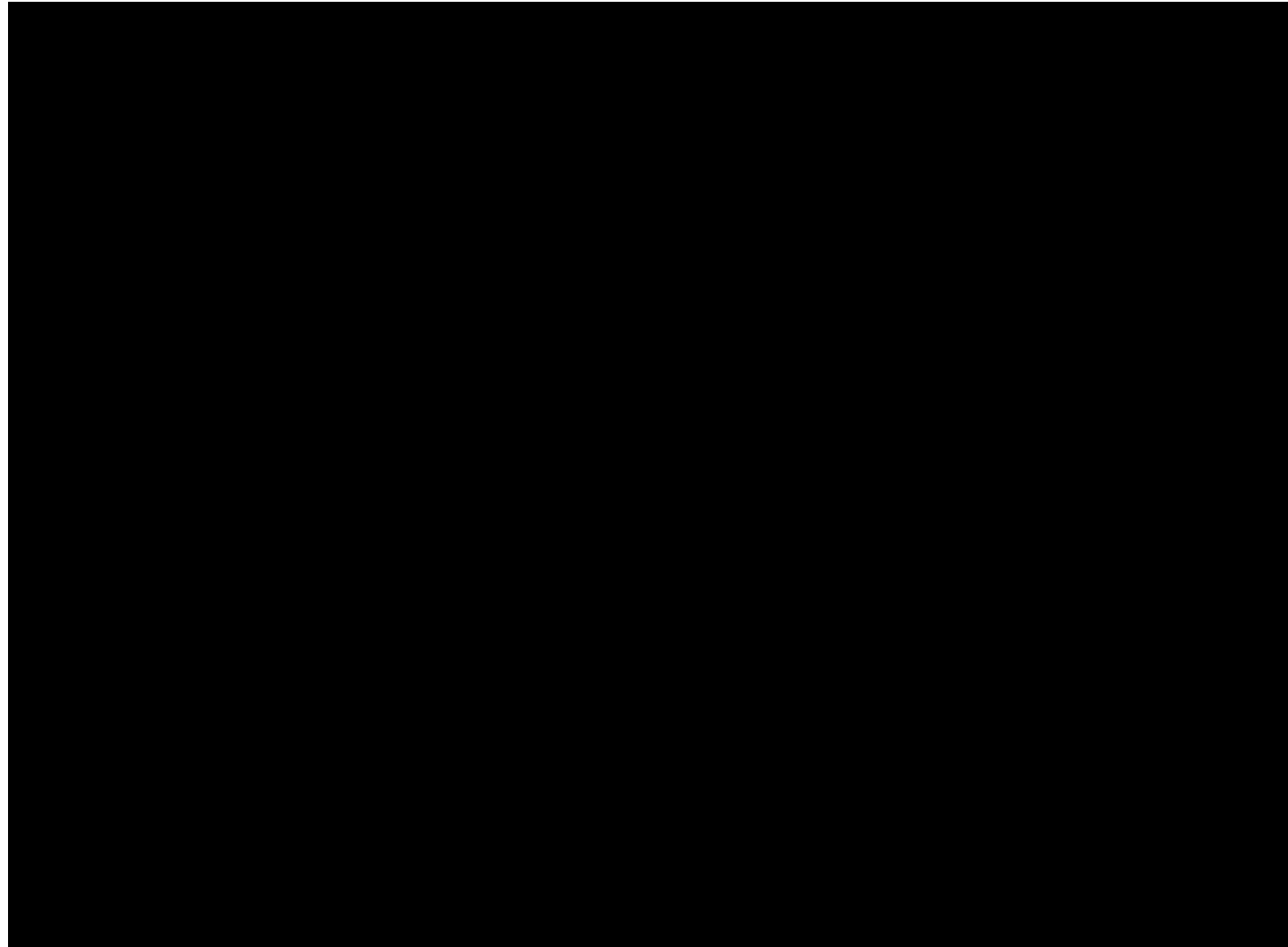
Stimulus presented



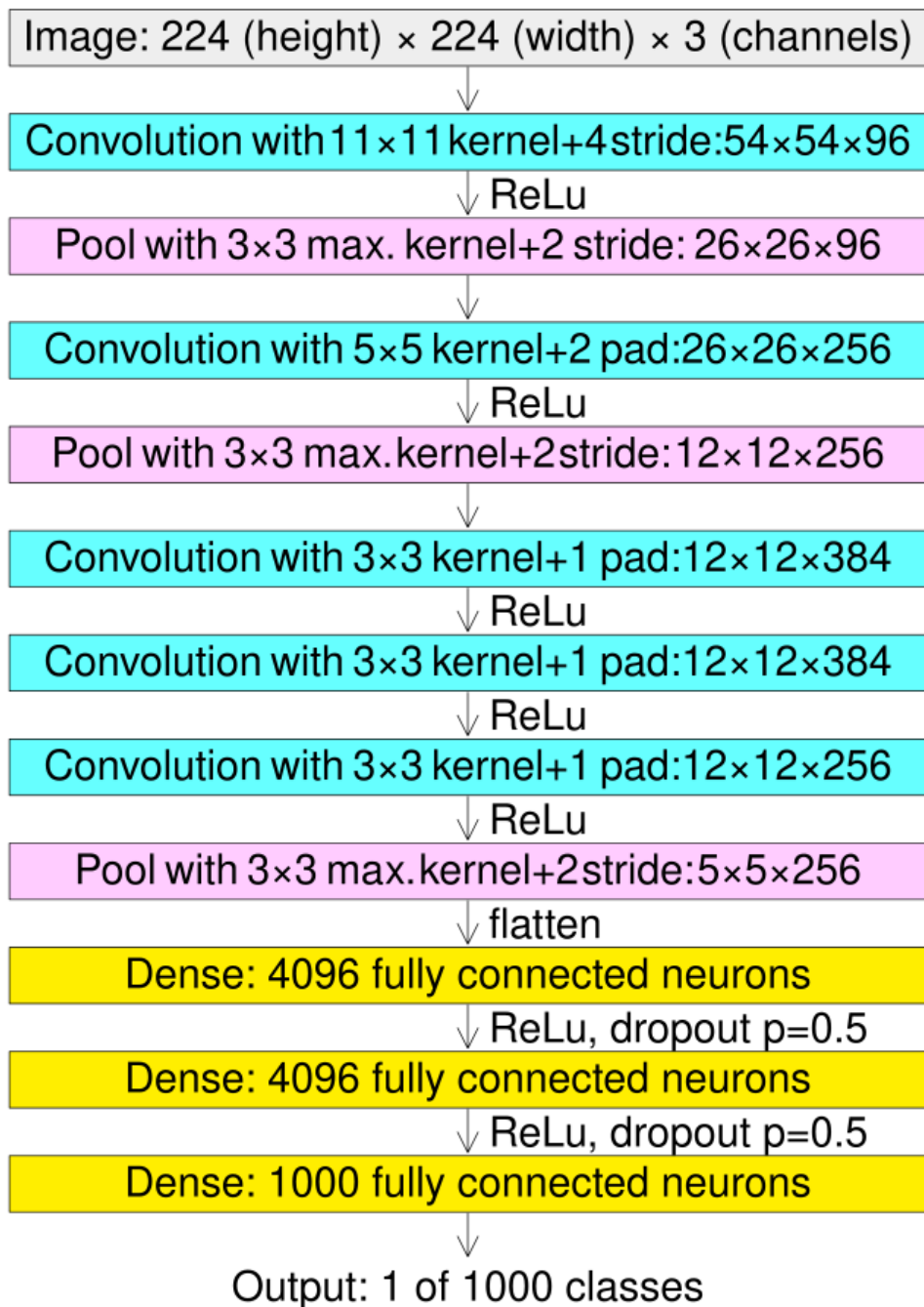
LeNet



Lenet test



AlexNet



Details/Retrospectives:

- first use of ReLU
- heavy data augmentation
- dropout 0.5
- batch size 128
- SGD Momentum 0.9

Historical note: Trained on GTX 580 GPU with only 3 GB of memory. Network spread across 2 GPUs, half the neurons (feature maps) on each GPU.

AlexNet implementation

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Dense, Flatten
```

```
model = Sequential()
```

```
# Layer 1: Convolutional layer with 64 filters of size 11x11x3
```

```
model.add(Conv2D(filters=64, kernel_size=(11,11), strides=(4,4), padding='valid', activation='relu',
input_shape=(224,224,3)))
```

```
# Layer 2: Max pooling layer with pool size of 3x3
```

```
model.add(MaxPooling2D(pool_size=(3,3), strides=(2,2)))
```

```
# Layer 3-5: 3 more convolutional layers with similar structure as Layer 1
```

```
model.add(Conv2D(filters=192, kernel_size=(5,5), padding='same', activation='relu'))
```

```
model.add(MaxPooling2D(pool_size=(3,3), strides=(2,2)))
```

```
model.add(Conv2D(filters=384, kernel_size=(3,3), padding='same', activation='relu'))
```

```
model.add(Conv2D(filters=256, kernel_size=(3,3), padding='same', activation='relu'))
```

```
model.add(MaxPooling2D(pool_size=(3,3), strides=(2,2)))
```

```
# Layer 6: Fully connected layer with 4096 neurons
```

```
model.add(Flatten())
```

```
model.add(Dense(4096, activation='relu'))
```

```
# Layer 7: Fully connected layer with 4096 neurons
```

```
model.add(Dense(4096, activation='relu'))
```

```
model.add(Dense(1000, activation='softmax'))
```

AlexNet implementation

```
class AlexNet(nn.Module):
    def __init__(self, num_classes=10):
        super(AlexNet, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(3, 96, kernel_size=11, stride=4, padding=0),
            nn.BatchNorm2d(96),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size = 3, stride = 2))
        self.layer2 = nn.Sequential(
            nn.Conv2d(96, 256, kernel_size=5, stride=1, padding=2),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size = 3, stride = 2))
        self.layer3 = nn.Sequential(
            nn.Conv2d(256, 384, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(384),
            nn.ReLU())
        self.layer4 = nn.Sequential(
            nn.Conv2d(384, 384, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(384),
            nn.ReLU())
        self.layer5 = nn.Sequential(
            nn.Conv2d(384, 256, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size = 3, stride = 2))
        self.fc = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(9216, 4096),
            nn.ReLU())
        self.fc1 = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
            nn.ReLU())
        self.fc2 = nn.Sequential(
            nn.Linear(4096, num_classes))
```

```
def forward(self, x):
    out = self.layer1(x)
    out = self.layer2(out)
    out = self.layer3(out)
    out = self.layer4(out)
    out = self.layer5(out)
    out = out.reshape(out.size(0), -1)
    out = self.fc(out)
    out = self.fc1(out)
    out = self.fc2(out)
    return out
```